

Design Swiggy / Zomato

Swiggy · Zomato · Uber Eats · DoorDash – భారతదేశ interviews lo **అత్యధికంగా అడిగే** problem, ఎందుకంటే ప్రతి ఒక్కరూ దీన్ని వాడతారు.

మూడు విరుపులు. మొదటిది Ravi చెల్లించిన ₹250 ని **₹300 గా మార్చేస్తుంది**. రెండోది 2,000 searches కి **8.5 సెకన్లు** తీసుకుంటుంది. మూడోది ఒక order ని **ఒకసారి cancelled మరియు delivered** చేస్తుంది.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి output, ప్రతి సంఖ్య నిజంగా `node` `lo run` చేసినదే – సమయాలతో సహా.

ఈ PROBLEM ఎందుకు ముఖ్యం

Infosys, TCS, Wipro, Accenture నుంచి Swiggy, Zomato, Zepto వరకు – ఇది అందరూ అడిగేది, ఎందుకంటే domain ని వివరించాల్సిన అవసరం లేదు. అందరికీ తెలుసు.

అందుకే ఇది కష్టమైనది కూడా: domain తెలుసు కాబట్టి interviewer నేరుగా design మీదకి వస్తాడు, మరియు మీరు ఎంత లోతుగా ఆలోచిస్తారో చూస్తాడు. "Order ఒక class, Restaurant ఒక class" అని చెప్పే సరిపోదు.

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు ఇందులో దాగిన మూడు వేర్వేరు systems
2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

Part 2 – మొదటి విరుపు: గతాన్ని తిరగరాయడం

3. Step – Order ఒక MenuItem ని పట్టుకుంటుంది
4. మొదటి విరుపు – restaurant ధర పెంచితే Ravi bill కూడా పెరిగింది
5. Step – Order ఒక గత సంఘటన

Part 3 – రెండో విరుపు: "దగ్గరలో ఏమున్నాయి?"

6. Step – అన్ని restaurants ని చూసి దూరం లెక్కించడం
7. రెండో విరుపు – 2,000 searches కి 8.5 సెకన్లు
8. Step – నగరాన్ని గడులుగా విభజించడం · మరియు అందులోని ఉచ్చు

Part 4 – మూడో విరుపు: order ఎక్కడ ఉంది?

9. Step – ఐదు boolean flags
10. మూడో విరుపు – ఒకేసారి cancelled మరియు delivered
11. Step – ఒక స్థితి + అనుమతించిన మార్పుల పట్టిక

Part 5 – పూర్తి system

12. Step – బిల్లు · ఒక పెద్ద సూత్రం కాదు, నియమాల జాబితా
13. మొత్తం code ఒకే చోట · నడిపి చూద్దాం

Part 6 – Interview lo

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి
15. నోటితో చెప్పాల్సిన English script
16. Follow-ups – tracking, scale, ratings
17. ఏమి నేర్చుకున్నాం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు ఇందులో దాగిన మూడు వేర్వేరు systems

"Design a food delivery system like Swiggy. Users browse nearby restaurants, place orders, and track delivery."

ఈ ఒక్క వాక్యంలో మూడు పూర్తిగా వేర్వేరు systems ఉన్నాయి:

ఒక APP • మూడు వేర్వేరు సమస్యలు

"Swiggy design చెయ్యి" – లోపల ఏమి ఉంది

1 • వెతకడం "దగ్గరలో ఏమున్నాయి?" ఇది ఒక ప్రాథమిక సమస్య – database సమస్య కాదు → ₹7 • 8.5 సెకన్లు	2 • Order మరియు డబ్బు ధరలు, coupons, GST, refunds ఒక్క రూపాయి తప్పితే అది నిజమైన సమస్య → ₹4 • ₹250 → ₹300	3 • Order ప్రయాణం placed → delivered cancel ఎప్పుడు చెల్లుతుంది? refund ఎంత? → ₹10 • ₹200 సెకన్లు
---	--	--

చాలా మంది ఇక్కడే అగిపోతారు

వారు నేరుగా classes గీస్తారు – User, Restaurant, MenuItem, Order, DeliveryPartner –
మరియు అది సరైనదే. కానీ అది ఒక జాబితా, ఒక design కాదు.

Interviewer చూసేది: ఈ classes మధ్య ఏమి తప్పు కావచ్చు అని మీరు ఆలోచించారా.

ఈ doc lo ఆ మూడు "ఏమి తప్పు కావచ్చు" అనే అడుగు అడుగునా విరగొట్టి, సరిచేస్తాం.

2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

ప్రశ్న	జవాబు నా design ని ఎలా మారుస్తుంది
Restaurant ధర మార్చితే పాత orders ఏమవుతాయి?	ఇదే ₹4 – ఈ ప్రశ్న అడగడమే సగం జవాబు

ప్రశ్న	జవాబు నా design ని ఎలా మారుస్తుంది
"దగ్గరలో" అంటే ఎంత దూరం – మరియు ఎన్ని restaurants ఉంటాయి?	20,000 నా 2,00,000 నా – §7 lo తేడా చూస్తాం
Order ని ఎప్పటివరకు cancel చేయొచ్చు?	ఇది §11 – refund నియమాలని నిర్ణయిస్తుంది
Delivery fee ఎలా లెక్కిస్తారు – దూరం, surge, coupons?	§12 – ఒక సూత్రమా, నియమాల జాబితానా
Live tracking కావాలా?	అవును అంటే పూర్తిగా వేరే system (§16)
ఒక partner ఒకేసారి ఎన్ని orders?	Batching – §16
Ratings, recommendations?	వదిలేయమని అడగండి – ఇది వేరే problem

మొదటి ప్రశ్న INTERVIEW ని మలుపు తిప్పుతుంది

"Restaurant ఒక item ధరని ₹250 నుంచి ₹300 కి మార్చింది. నిన్న ₹250 కి order చేసిన Ravi యొక్క order history ఇప్పుడు ఏమి చూపించాలి?"

జవాబు స్పష్టంగా **₹250**. అతను చెల్లించింది అదే.

కానీ మీరు Order lo MenuItem ని నేరుగా పట్టుకుంటే – అది **₹300** చూపిస్తుంది. మరియు మీరు దాన్ని ఎప్పటికీ గమనించరు, ఎందుకంటే test lo ధర మారదు.

§4 lo దాన్ని నడిపి చూద్దాం.

Part 2 – మొదటి విరుపు: గతాన్ని తిరగరాయడం

3. Step – Order ఒక MenuItem ని పట్టుకుంటుంది

అతి సహజమైన మొదటి version. Restaurant కి ఒక menu, Order lo ఆ items:

```
class MenuItem {
  constructor(name, price) { this.name = name; this.price = price; }
}

class Order {
  constructor(id, restaurant, items) { // items = [{ item, qty }]
    this.id = id; this.restaurant = restaurant; this.items = items;
  }
  get total() {
    return this.items.reduce((s, l) => s + l.item.price * l.qty, 0);
  }
  describe() {
    return this.items.map(l => `${l.qty}x ${l.item.name} @₹${l.item.price}`)
      .join(', ');
  }
}
```

ఇది చదవడానికి బాగుంది. Order ఒక MenuItem ని సూచిస్తోంది – **నకలు తయారేదు**, ఇది మంచి పద్ధతిలా అనిపిస్తుంది. Ravi order చేద్దాం:

```
const biryaniHouse = new Restaurant('Biryani House')
  .addItem(new MenuItem('Chicken Biryani', 250))
  .addItem(new MenuItem('Raita', 40));

const o1 = new Order('ORD-1', biryaniHouse, [
  { item: biryaniHouse.item('Chicken Biryani'), qty: 1 },
  { item: biryaniHouse.item('Raita'), qty: 1 },
]);
```

Ravi order చేసినప్పుడు:
1x Chicken Biryani @₹250, 1x Raita @₹40
మొత్తం: ₹290

సరిగ్గానే ఉంది. ఇప్పుడు **మరుసటి రోజు** restaurant ధర పెంచింది.

4. మొదటి విరుపు – restaurant ధర పెంచితే Ravi bill కూడా పెరిగింది

```
biryaniHouse.item('Chicken Biryani').price = 300; // ధర పెంచారు
```

మనం Ravi order ని తాకనలేదు. అయినా:

```
మరుసటి రోజు, restaurant ధర పెంచాక – *అదే పాత* order:  
1x Chicken Biryani @₹300, 1x Raita @₹40  
మొత్తం: ₹340
```

ఇంకొంచెం ముందుకే వెళదాం – పేరు మార్చి, Raita ని menu నుంచే తీసేద్దాం:

```
biryaniHouse.item('Chicken Biryani').name = 'Chicken Dum Biryani (New!)';  
biryaniHouse.menu.delete('Raita');
```

```
Raita ని menu నుంచి తీసేసి, biryani పేరు మార్చాక:  
1x Chicken Dum Biryani (New!) @₹300, 1x Raita @₹40  
మొత్తం: ₹340
```

రెండో పంక్తి ఒక విచిత్రం: **Raita menu lo లేదు, కానీ order అది ఇంకా ₹40 అని చెప్తోంది.** ఎందుకంటే order ఆ వస్తువుని పట్టుకుని ఉంది. అంటే order కొన్ని విషయాల్లో గతాన్ని నిలుపుతోంది, కొన్నింటిలో నిలవట్లేదు – **అదే అసలు ప్రమాదం:** ఏది మారుతుందో, ఏది మారదో ఊహించడం సాధ్యం కాదు.

మొదటి విరుపు – ORDER తనంతట తానే మారిపోయింది

Ravi ₹290 చెల్లించాడు. అతని order history ఇప్పుడు ₹340 చూపిస్తోంది. అతను ఎప్పుడూ order చేయని పేరున్న item ని చూపిస్తోంది.

ఇది కేవలం UI తప్పు కాదు – ఇది డబ్బు:

```
Ravi చెల్లించింది : ₹250  
ఒక వారం తర్వాత total : ₹300  
Ravi "refund" అడిగితే : ₹300 ← ₹50 ఎక్కువ
```

Refund ₹300 వెళుతుంది. కంపెనీకి ₹50 నష్టం, ప్రతి అలాంటి order కీ. మరియు accounting ఎప్పుడూ సరిపోదు – ఎందుకంటే నిన్నటి మొత్తాలు ఈరోజు ధరల ప్రకారం మారుతున్నాయి.

అసలు సమస్య: Order అంటే ఏమిటి?

ఈ bug ఒక పొరపాటు కాదు – ఇది ఒక భావనాపరమైన తప్పు.

మనం `Order` ని ఇప్పుడు ఉన్న దాని గురించిన ఒక view గా రాశాం. కానీ order అనేది అది కాదు.

Order అనేది ఒక గత సంఘటన యొక్క రికార్డు. అది "Ravi ఏమి కొంటాడు" అని చెప్పదు – "Ravi ఫలానా రోజు, ఫలానా ధరకీ, ఫలానా పేరున్న దాన్ని కొన్నాడు" అని చెప్తుంది.

గత సంఘటనలు **మారవు**. కాబట్టి order lo ఉన్న డేటా కూడా మారకూడదు.

ఇది రెండు విషయాల మధ్య తేడా: **సూచన** (reference – "ఆ item ని చూడు") మరియు **ప్రతిబింబం** (snapshot – "ఆ క్షణంలో ఆ item ఇలా ఉంది"). Menu కి సూచన సరైనది; order కి ప్రతిబింబం సరైనది.

Deep Dive 04 (BookMyShow) §5 lo ఒక దగ్గరి బంధువుని చూశాం – అక్కడ ఒక **Seat** వస్తువు రెండు shows lo పంచుకోబడింది. అక్కడ సమస్య ఒకే వస్తువు పలు చోట్లు; ఇక్కడ సమస్య ఒక వస్తువు కాలంతో పాటు మారడం.

5. Step – Order ఒక గత సంఘటన

ఆ ఆలోచనని ఒక class lo పెడదాం. **OrderLine** – order చేసిన క్షణంలోని విలువల ప్రతిబింబం:

```
class OrderLine {
  constructor(item, qty) {
    this.itemId = item.name; // ఏ item అని గుర్తించడానికి
    this.name = item.name; // ఆ క్షణంలోని పేరు
    this.price = item.price; // ఆ క్షణంలోని ధర
    this.qty = qty;
    Object.freeze(this); // ఇక ఎవరూ మార్చలేరు
  }
  get amount() { return this.price * this.qty; }
}
```

రెండు కీలక వివరాలు:

- itemId మరియు name విడివిడిగా ఉన్నాయి.** **itemId** అనేది "ఏ item" (ఇది మారకూడదు, reports కి కావాలి); **name** అనేది "ఆ క్షణంలో అది ఎలా కనిపించింది" (ఇది receipt కి కావాలి).
- Object.freeze** – ఇది ఒక వ్యాఖ్య కాదు, ఒక **అమలు**. ఎవరైనా మార్చడానికి ప్రయత్నిస్తే code ఆగిపోతుంది.

```
class Order {
  constructor(id, restaurantName, lines) {
    this.id = id; this.restaurantName = restaurantName; this.lines = lines;
    Object.freeze(this.lines);
  }
  get total() { return this.lines.reduce((s, l) => s + l.amount, 0); }
}
```

restaurant కూడా ఒక వస్తువు కాదు – **restaurantName**, ఒక ప్రతిబింబం. Restaurant పేరు మార్చుకున్నా, పాత receipt పాత పేరే చూపించాలి.

అదే పరీక్ష మళ్ళీ:

Ravi order చేసినప్పుడు:

1× Chicken Biryani @₹250, 1× Raita @₹40 → ₹290

ధర పెంచి, పేరు మార్చాక – అదే order:

1× Chicken Biryani @₹250, 1× Raita @₹40 → ₹290 ← మారలేదు ✓

order line ని మార్చే ప్రయత్నం: Cannot assign to read only property 'price' of object '#<OrderLine>'

order line ధర ఇప్పటికీ: ₹250

₹290 ₹290 గానే ఉంది, మరియు మార్చే ప్రయత్నం ఆగిపోయింది.

ఇది ఒక సార్వత్రిక నియమం – మరియు INTERVIEW LO చెప్పడగినది

"ఒక గత సంఘటనని నమోదు చేసే ప్రతి రికార్డు – order, invoice, payment, audit log – ఆ క్షణంలోని విలువలని ప్రతిబింబంగా నిల్వ చేయాలి, సూచనగా కాదు."

Amazon invoice, bank statement, salary slip – అన్నీ ఇలాగే పనిచేస్తాయి. మీ కంపెనీ పేరు మారినా, మీ పాత salary slip పాత పేరే చూపిస్తుంది.

ఇది ఒక వాక్యంలో చెప్పగలిగితే, interviewer కి మీరు **నిజమైన systems** గురించి ఆలోచిస్తున్నారని తెలుస్తుంది.

Part 3 – రెండో విరుపు: "దగ్గరలో ఏమున్నాయి?"

6. Step – అన్ని restaurants ని చూసి దూరం లెక్కించడం

ఇప్పుడు రెండో system: "నా దగ్గర 2 km లోపల ఏమున్నాయి?"

సహజమైన ఆలోచన: అన్నిటి దూరం లెక్కించి, 2 km లోపు వాటిని ఉంచడం.

```
const KM_PER_DEG_LAT = 111.0;
const kmPerDegLon = (lat) => 111.320 * Math.cos(lat * Math.PI / 180);

function distanceKm(a, b) {
  const dy = (a.lat - b.lat) * KM_PER_DEG_LAT;
  const dx = (a.lon - b.lon) * kmPerDegLon(a.lat);
  return Math.hypot(dx, dy);
}

class NaiveDirectory {
  constructor() { this.all = []; }
  add(r) { this.all.push(r); }
  nearby(point, radiusKm) {
    return this.all.filter(r => distanceKm(point, r) <= radiusKm);
  }
}
```

ఆ kmPerDegLon ఎందుకు? ఒక డిగ్రీ అక్షాంశం (latitude) ఎక్కడైనా ~111 km. కానీ ఒక డిగ్రీ రేఖాంశం (longitude) భూమధ్యరేఖ వద్ద ~111 km, ధ్రువాల వద్ద **సున్నా**. Hyderabad (17.4°N) వద్ద అది ~106 km. దీన్ని విస్మరిస్తే తూర్పు-పడమర దూరాలు ~5% తప్పుగా వస్తాయి – మరియు §8 lo ఇదే ఒక ఉచ్చుగా మారుతుంది.

ఇది పనిచేస్తుంది. మరి సమస్య ఏమిటి? – **కొలిచి చూద్దాం.**

7. రెండో విరుపు – 2,000 searches కి 8.5 సెకన్లు

40 km × 40 km నగరం, restaurants యాదృచ్ఛికంగా పరచబడ్డాయి, 2,000 users "2 km లోపల" వెతుకుతున్నారు:

```
2,000 searches · "2 km లోపల" · అన్నిటిని scan చేస్తే

1,000 restaurants :      53 ms
```

20,000 restaurants : 907 ms
200,000 restaurants : 8,585 ms

అన్నిటిని SCAN చేయడం • 2,000 SEARCHES

Restaurants 200x పెరిగితే, సమయం 162x పెరిగింది - ఇది సరళ రేఖ

1,000 53 ms

20,000 907 ms

200,000 8,585 ms - ఒక్క batch searches కి 8.5 సెకన్లు

ఇది $O(n)$: ప్రతి search ప్రతి restaurant ని చూస్తుంది, అది 40 km దూరంలో ఉన్నా.
Delhi lo ఒక restaurant చేరితే, Hyderabad lo ప్రతి search **నెమ్మదవుతుంది**. అది అర్థరహితం.

రెండో విరుపు - దూరం ఉన్నవాటిని కూడా చూస్తున్నాం

Hyderabad lo ఉన్న Ravi కి 2 km లోపల ఏమున్నాయో తెలియాలంటే - code **Delhi, Mumbai, Chennai lo ఉన్న ప్రతి restaurant** దూరాన్ని లెక్కిస్తోంది, ఆపై వాటిని వదిలేస్తోంది.

మౌలిక తప్పు: restaurants ఎక్కడ ఉన్నాయో అనే సమాచారాన్ని మనం నిల్వలో వాడట్లేదు - కేవలం filter lo వాడుతున్నాం. స్థానం ఒక లక్షణం మాత్రమే అయిపోయింది, ఒక సూచిక(index) కాలేదు.

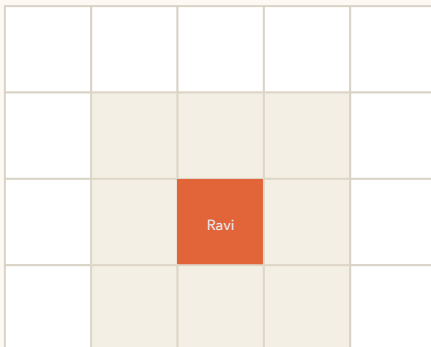
ఇది Deep Dive 08 (HashMap) §5 lo చూసిన అదే ఆలోచన: **వెతకొద్దు - ఎక్కడ ఉందో లెక్కించు.**

8. Step - నగరాన్ని గడులుగా విభజించడం • మరియు అందులోని ఉచ్చు

నగరాన్ని 2 km x 2 km **గడులుగా** విభజిద్దాం. ప్రతి restaurant ఒక గడిలో ఉంటుంది. "2 km లోపల" అడిగితే - **చుట్టపక్కల 9 గడులు మాత్రమే** చూస్తే సరిపోతుంది.

గడులు • 2 KM RADIUS కి 3x3 గడులు చాలు

నగరం మొత్తం 400 గడులు - కానీ ఒక search 9 గడులని మాత్రమే తాకుతుంది



ప్రతి గడి 2 km x 2 km

Radius 2 km అంటే - ఏ restaurant ఐనా పక్క గడిని దాటి ఉండదు

కాబట్టి 3 x 3 = 9 గడులు చాలు

మిగతా 391 గడులని **అసలు తెరవనే లేదు**

Delhi lo restaurant చేరినా

ఈ 9 గడులు మారవు - అదే అసలు లాభం

```
class GridDirectory {  
    constructor(cellKm = 2, refLat = 17.385) {  
        this.cellKm = cellKm;  
    }  
}
```

```

    this.lonScale = kmPerDegLon(refLat); // ← సగరానికి ఒకే scale (కింద చూడండి)
    this.cells    = new Map();           // "row:col" → [restaurants]
  }
  #key(lat, lon) {
    const row = Math.floor(lat * KM_PER_DEG_LAT / this.cellKm);
    const col = Math.floor(lon * this.lonScale / this.cellKm);
    return `${row}:${col}`;
  }
  add(r) {
    const k = this.#key(r.lat, r.lon);
    if (!this.cells.has(k)) this.cells.set(k, []);
    this.cells.get(k).push(r);
  }
  nearby(point, radiusKm) {
    const span = Math.ceil(radiusKm / this.cellKm); // ఎన్ని గడులు పక్కకి
    const r0 = Math.floor(point.lat * KM_PER_DEG_LAT / this.cellKm);
    const c0 = Math.floor(point.lon * this.lonScale / this.cellKm);
    const out = [];
    for (let dr = -span; dr <= span; dr++)
      for (let dc = -span; dc <= span; dc++)
        for (const r of this.cells.get(`${r0+dr}:${c0+dc}`) ?? [])
          if (distanceKm(point, r) <= radiusKm) out.push(r);
    return out;
  }
}

```

పోల్చి చూద్దాం – మరియు ముందు సరైనవేనా చూద్దాం

వేగం అర్థవంతం కావాలంటే ముందు జవాబు సరైనదై ఉండాలి. కాబట్టి 300 searches కి రెండు పద్ధతుల ఫలితాలని పోల్చుదాం:

2,000 searches · "2 km లోపల"

restaurants	అన్నీ scan	గడులు (2 km)	వేగం
1000	53 ms	4 ms	13×
[రెండు ఒకే ఫలితాలు ఇస్తున్నాయా? అవును ✓]			
20000	907 ms	22 ms	41×
200000	8585 ms	418 ms	21×

907 ms → 22 ms. మరియు ఫలితాలు అక్షరాలా ఒకటే.

ఈ సంఖ్యలు ఎంత స్థిరమైనవి? పలుమార్లు నడిపితే scan వరుస 53-56 ms, 890-915 ms, 8,490-8,600 ms; గడుల వరుస 3-4 ms, 20-22 ms, 418-446 ms. అంటే రెండు పెద్ద వరుసల **వేగం 41-45×** మరియు **19-21×** మధ్య ఉగుతుంది. 1,000 restaurants వరుసని అస్సలు నమ్మొద్దు – 3 నా 4 నా అనే తేడా ఆ 13× ని 18× చేస్తుంది. **అంత చిన్న సమయాలు కొలవడం అర్థరహితం;** ఆ వరుస ఉన్నది ఆకారం చూపించడానికే.

ఒక ముఖ్యమైన గమనిక: గడి కూడా పెరుగుతోంది – ఎందుకు?

గడుల వరుస కూడా 4 → 22 → 418 ms గా పెరిగింది. అది తప్పు కాదు, **ఆశించినదే:** నగరం పరిమాణం స్థిరంగా ఉంచి restaurants ని 200× పెంచాం, కాబట్టి ప్రతి గడిలో **200× ఎక్కువ** ఉన్నాయి.

గడల ఖర్చు మొత్తం సంఖ్య మీద ఆధారపడదు – స్థానిక దట్టత మీద ఆధారపడుతుంది.
నిజమైన దేశవ్యాప్త index lo Delhi lo restaurants చేరితే Hyderabad search అస్సలు
నెమ్మదించదు. Scan పద్ధతిలో నెమ్మదిస్తుంది. అదే అసలు తేడా.

గడి పరిమాణం ఎంత ఉండాలి? – కలిచి చూద్దాం

ఇది ఒక నిజమైన ఎంపిక, మరియు తప్పు చేస్తే లాభం చాలా తగ్గుతుంది:

వ్యాసార్థం 1 km	(సగటున 384 ఫలితాలు)
అన్నీ scan	: 9774 ms
గడి 1 km	: 116 ms (84×) గడులు=1587
గడి 2 km	: 401 ms (24×) గడులు=419
వ్యాసార్థం 2 km	(సగటున 1499 ఫలితాలు)
అన్నీ scan	: 9261 ms
గడి 2 km	: 417 ms (22×) గడులు=419
గడి 4 km	: 1450 ms (6×) గడులు=121
వ్యాసార్థం 3 km	(సగటున 3302 ఫలితాలు)
అన్నీ scan	: 8598 ms
గడి 3 km	: 893 ms (10×) గడులు=196
గడి 6 km	: 3420 ms (3×) గడులు=49

ఈ పట్టికలో "2 km · అన్నీ scan" **9,261 ms** అని ఉంది, కానీ పై పట్టికలో అదే కాలం **8,585 ms**. రెండూ నిజమైన runs – ఇవి వేర్వేరు ప్రయోగాలు, వేర్వేరుగా తయారైన డేటాతో, మరియు ఒకే process lo వరుసగా నడిచినవి. **~8% తేడా** అనేది ఇలాంటి కాలంతల్లో సాధారణం.

పాఠం: benchmark సంఖ్యలని నిష్పత్తులుగా చదవండి, ఖచ్చితమైన విలువలుగా కాదు. "8,585 ms" ముఖ్యం కాదు; "22×" ముఖ్యం.

ప్రతి జతలోనూ **గడి = వ్యాసార్థం** అయినప్పుడు 3-4 రెట్లు వేగం. ఎందుకంటే గడిని రెట్టింపు చేస్తే – అదే 3×3 గడులు, కానీ **నాలుగు రెట్లు వైశాల్యం**, అంటే నాలుగు రెట్లు అభ్యర్థులు.

నియమం: గడి పరిమాణం $\approx$ సాధారణ search వ్యాసార్థం. చిన్నది చేస్తే ఎక్కువ గడులు తెరవాలి; పెద్దది చేస్తే ఒక్కో గడిలో ఎక్కువ వృధా.

ఇప్పుడు ఆ ఉచ్చు – నేను దీన్నో పడ్డాను

మొదట నేను `#key` ని ఇలా రాశాను:

```
const col = Math.floor(lon * kmPerDegLon(lat) / this.cellKm); // ← తప్పు
```

అది సహజంగా అనిపిస్తుంది – ఆ point యొక్క అక్షాంశం వద్ద ఉన్న సరైన scale వాడుతున్నాం కదా? కానీ ఫలితం:

రెండూ ఒకే ఫలితాలు ఇస్తున్నాయా? లేదు $\times$

ఎందుకు? `kmPerDegLon(lat)` అనేది **lat** మీద ఆధారపడుతుంది. అంటే ఒకే రేఖాంశంలో ఉన్న రెండు restaurants, కొంచెం ఉత్తరం-దక్షిణంగా ఉంటే, **వేర్వేరు నిలువు వరుసల్లో** పడతాయి. గడి ఒక సమతల జాలకం కాకుండా వంగిపోయింది – కొన్ని restaurants ఎప్పటికీ దొరకవు.

పరిష్కారం: నగరానికి **ఒక reference అక్షాంశం** వాడటం (`refLat`). అప్పుడు జాలకం సమతలంగా ఉంటుంది.

ఇది ఒక సాధారణ INDEX BUG యొక్క ఆకారం

Index కీ **స్థిరంగా** ఉండాలి. ఒక వస్తువు యొక్క కీ, ఆ వస్తువు యొక్క వేరే లక్షణం మీద ఆధారపడితే – index నమ్మదగినది కాదు.

Deep Dive 08 (HashMap) lo ఇదే నియమం వేరే రూపంలో వచ్చింది: **mutable key ని hash చేస్తే, key మారినప్పుడు entry మాయమవుతుంది**. ఇక్కడ కీ mutable కాదు, కానీ కీ లెక్కింపు స్థిరం కాదు – అదే ఫలితం.

మరియు ఇది ఎలా దొరికింది? – **వేగాన్ని కొలిచే ముందు సరైనతని పోల్చడం వల్ల**. ఆ పోలిక లేకపోతే నేను "12x వేగం!" అని సంతోషంగా రాసేవాడిని, తప్పు జవాబులతో.

Part 4 – మూడో విరుపు: order ఎక్కడ ఉంది?

9. Step – ఐదు boolean flags

మూడో system: order ప్రయాణం. Order placed అవుతుంది, restaurant accept చేస్తుంది, వండుతుంది, partner తీసుకుంటాడు, delivered.

సహజమైన ఆలోచన: ప్రతి దశకీ ఒక boolean.

```
class Order {
  constructor(id, amount) {
    this.id = id; this.amount = amount;
    this.isPlaced = true; this.isAccepted = false; this.isPickedUp = false;
    this.isDelivered = false; this.isCancelled = false;
  }
  accept() { this.isAccepted = true; }
  pickup() { this.isPickedUp = true; }
  deliver() { this.isDelivered = true; }
  cancel() { this.isCancelled = true; }

  refundDue() {
    if (this.isCancelled && !this.isPickedUp) return this.amount; // పూర్తి
    if (this.isCancelled && this.isPickedUp) return this.amount / 2; // సగం
    return 0;
  }
  get status() {
    if (this.isCancelled) return 'CANCELLED';
    if (this.isDelivered) return 'DELIVERED';
    if (this.isPickedUp) return 'PICKED_UP';
    if (this.isAccepted) return 'ACCEPTED';
    return 'PLACED';
  }
}
```

ఇది చాలా code bases lo నిజంగా కనిపించేది. ప్రతి method చిన్నది, స్పష్టమైనది. refundDue నియమాలు కూడా సహేతుకంగా ఉన్నాయి: తీసుకునే ముందు cancel అయితే పూర్తి refund, తర్వాత అయితే సగం.

10. మూడో విరుపు – ఒకేసారి cancelled మరియు delivered

నిజ జీవిత పరిస్థితి: partner భోజనం తీసుకున్నాడు. అదే సమయంలో Ravi app lo "Cancel" నొక్కాడు, మరియు partner "Delivered" నొక్కాడు.

```
const o = new Order('ORD-9', 400);
o.accept(); o.pickUp();

o.cancel(); // Ravi app lo "Cancel" నొక్కాడు
o.deliver(); // అదే సమయంలో partner "Delivered" నొక్కాడు
```

Partner biryani తీసుకున్నాడు : PICKED_UP

Ravi cancel + partner deliver తర్వాత:

```
status() : CANCELLED
isCancelled: true isDelivered: true
refundDue(): ₹200 ← భోజనం delivered అయినా సగం refund
```

ఏ దశా లేకుండా నేరుగా deliver:

```
status(): DELIVERED isAccepted: false isPickedUp: false
```

మూడో విరుపు - ORDER రెండు స్థితుల్లో ఒకేసారి ఉంది

1 • భోజనం తిన్నాక ₹200 refund. isCancelled మరియు isDelivered రెండూ true. Order నిజంగా delivered అయింది - Ravi తిన్నాడు - కానీ system ₹200 తిరిగి ఇచ్చింది.

2 • దశలు దాటేయడం. రెండో order restaurant accept చేయకుండా, ఎవరూ తీసుకోకుండా **DELIVERED** అయింది. ఇది ఒక API bug, ఒక tap రెండుసార్లు, ఒక retry - ఏదైనా చేయగలదు.

3 • status ఒక అబద్ధం. ఇది isCancelled ని ముందు చూస్తుంది కాబట్టి CANCELLED చెప్పింది. ఆ if ల క్రితం మార్చితే అదే order DELIVERED అవుతుంది. **ఒకే డేటా, రెండు జవాబులు.**

లెక్క వేసి చూద్దాం - ఎన్ని అసాధ్యమైన స్థితులు?

5 booleans → సాధ్యమైన కలయికలు: 32
వాటిలో నిజంగా అర్థవంతమైనవి : 5 (PLACED→ACCEPTED→PICKED_UP→DELIVERED, CANCELLED)
అంటే 27 కలయికలు *అసాధ్యమైనవి* - కానీ code వాటిని అనుమతిస్తోంది.

32 కలయికలు • 5 అర్థవంతమైనవి

5 booleans తో చెప్పగలిగేవి, vs నిజంగా ఉండగలిగేవి

code అనుమతించే స్థితులు: 32

5

నిజంగా ఉండగలిగినవి - మిగతా 27 కేవలం bugs కోసం ఖాళీ

నియమం: **అసాధ్యమైన స్థితిని వ్యక్తపరచడమే సాధ్యం కాకుండా చేయండి.**
Test తో bug ని పట్టుకోవడం కంటే, bug రాయడమే వీలుకాకుండా చేయడం మేలు.

Deep Dive 04 (BookMyShow) §9 lo ఒక దశ **తప్పిపోయింది** (payment మరియు booking మధ్య HELD). ఇక్కడ దశలు **ఉన్నాయి** - కానీ వాటి **మధ్య నియమాలు లేవు**. రెండు వేర్వేరు తప్పులు, ఒకే పరిష్కారం.

11. Step – ఒక స్థితి + అనుమతించిన మార్పుల పట్టిక

ఐదు booleans కి బదులు ఒక `state`, మరియు ఏ స్థితి నుంచి ఎక్కడికి వెళ్ళొచ్చే ఒక పట్టిక:

```
const TRANSITIONS = {
  PLACED:    ['ACCEPTED', 'REJECTED', 'CANCELLED'],
  ACCEPTED:  ['PREPARING', 'CANCELLED'],
  PREPARING: ['READY', 'CANCELLED'],
  READY:     ['PICKED_UP'],
  PICKED_UP: ['DELIVERED'],
  DELIVERED: [], // చివరి దశలు – ఇక్కడి నుంచి ఎక్కడికీ లేదు
  CANCELLED: [],
  REJECTED:  [],
};
```

ఈ పట్టికే మొత్తం **business నియమం**. దీన్ని చదివితే ఒక విషయం స్పష్టం: **PICKED_UP నుంచి CANCELLED కి దారి లేదు**. భోజనం బయలుదేరాక cancel లేదు – అది ఒక వాక్యం, ఒక `if` కాదు.

```
class Order {
  #state = 'PLACED';
  #history = [];
  constructor(id, amount) { this.id = id; this.amount = amount; }
  get state() { return this.#state; }
  get history() { return this.#history.map(h => h.to); }

  moveTo(next, at = Date.now()) {
    if (!TRANSITIONS[this.#state].includes(next))
      return { ok: false, reason: `${this.#state} → ${next} అనుమతి లేదు` };
    this.#history.push({ from: this.#state, to: next, at });
    this.#state = next;
    return { ok: true, state: next };
  }

  refundDue() {
    if (this.#state === 'REJECTED') return this.amount;
    if (this.#state !== 'CANCELLED') return 0; // DELIVERED తో సహా
    return this.#history.some(h => h.to === 'PICKED_UP')
      ? this.amount / 2 : this.amount;
  }
}
```

మూడు వివరాలు గమనించండి:

- **#state ప్రైవేట్**. బయటి నుంచి `order.state = 'DELIVERED'` రాయడం వీలుకాదు. ఒకే ద్వారం – `moveTo`.
- **#history** ఉంది. ఇది కేవలం `log` కాదు – `refundDue` దాన్ని వాడుతుంది ("ఎప్పుడైనా PICKED_UP అయిందా?"). Support team కి కూడా ఇదే కావాలి.
- **తిరస్కరణ ఒక వస్తువు**, `throw` కాదు. UI "PICKED_UP → CANCELLED అనుమతి లేదు" అని నేరుగా చూపించగలదు.

అదే పరిస్థితి మళ్ళీ:

Partner biryani తీసుకున్నాడు: PICKED_UP

Ravi "Cancel" స్కీత్ : { ok: false, reason: 'PICKED_UP → CANCELLED అనుమతి లేదు' }

Partner "Delivered" : { ok: true, state: 'DELIVERED' }

చివరి స్థితి : DELIVERED refund: ₹0

దశలు దాటేయడం: { ok: false, reason: 'PLACED → DELIVERED అనుమతి లేదు' }

స్థితి ఇప్పటికీ: PLACED

వండుతుండగా cancel: CANCELLED → refund ₹500 (పూర్తి)

ప్రయాణం: ACCEPTED → PREPARING → CANCELLED

మూడూ సరైనవి. Cancel **తిరస్కరించబడింది** మరియు కారణం చెప్పింది; delivery కొనసాగింది; refund **₹0**.
దశలు దాటడం ఆగిపోయింది. మరియు వండుతుండగా cancel చేస్తే **పూర్తి refund** – ఎందుకంటే ఆ order
ఎప్పుడూ **PICKED_UP** కాలేదు, మరియు **#history** కి అది తెలుసు.

Part 5 – పూర్తి system

12. Step – బిల్లు • ఒక పెద్ద సూత్రం కాదు, నియమాల జాబితా

మిగిలించి ఒక్కటే: **డబ్బు లెక్క** సహజమైన రూపం ఇది:

```
const total = itemTotal + packaging + deliveryFee + surge
               + (itemTotal * 0.05) - coupon;    // ← ఒక పెద్ద సూత్రం
```

ఇది పనిచేస్తుంది – కానీ app **ఇది చూపించదు**. App ఒక **విభజించిన బిల్లు** చూపిస్తుంది, ప్రతి వరుసకీ ఒక పేరుతో. మరియు అదే నిజమైన అవసరం: మొత్తం ఎందుకు ₹331 అయిందో user కి తెలియాలి, support team కి తెలియాలి.

కాబట్టి ప్రతి charge ని ఒక **వస్తువు** చేద్దాం, మరియు ప్రతిదీ ఒక **పేరున్న వరుస** ఇస్తుంది:

```
class DistanceFee {
  constructor(base = 20, perKm = 8) { Object.assign(this, { base, perKm }); }
  apply(ctx) {
    return { label: `Delivery (${ctx.distanceKm.toFixed(1)} km)`,
             amount: this.base + Math.ceil(ctx.distanceKm) * this.perKm };
  }
}

class FlatCoupon {
  constructor(code, off, minOrder = 0) {
    Object.assign(this, { code, off, minOrder });
  }
  apply(ctx) {
    if (ctx.itemTotal < this.minOrder) return null; // వర్తించదు – వరుస లేదు
    return { label: `Coupon ${this.code}`, amount: -this.off };
  }
}
```

`null` తిరిగివ్వడం ఒక చిన్న వివరం కానీ ముఖ్యమైనది: **వర్తించని charge కి బిల్లులో వరుస ఉండదు. ₹0 వరుస ఉండదు.**

Bill అన్నిటినీ **క్రమంలో** నడిపిస్తుంది:

```
class Bill {
  constructor(lines, distanceKm, charges) {
    this.lines = lines;
    this.itemTotal = lines.reduce((s, l) => s + l.amount, 0);
    this.rows = [];
    let runningTotal = this.itemTotal;
    for (const c of charges) {
      const row = c.apply({ lines, distanceKm,
```

```

        itemTotal: this.itemTotal, runningTotal });
    if (!row) continue;
    this.rows.push(row);
    runningTotal += row.amount;
  }
  this.total = Math.round(runningTotal * 100) / 100;
  Object.freeze(this);
}
}

```

మరియు ఇక్కడ ఒక ఊహించని విషయం: క్రమం కూడా డబ్బు

charges ఒక **జాబితా**, అంటే దానికి ఒక క్రమం ఉంది. GST ని coupon కి ముందు వేయాలా, తర్వాత? రెండూ సహేతుకంగా అనిపిస్తాయి. కొలిచి చూద్దాం:

```

--- నియమాల *క్రమం* కూడా ఒక business నిర్ణయం ---
GST ముందు, coupon తర్వాత : ₹331.15
Coupon ముందు, GST తర్వాత : ₹328.65
తేడా : ₹2.50 → రోజుకి 1,00,000 orders అంటే ₹2,50,000

```

ఒక order కి ₹2.50. రోజుకి లక్ష orders అంటే **₹2,50,000** – రోజుకి.

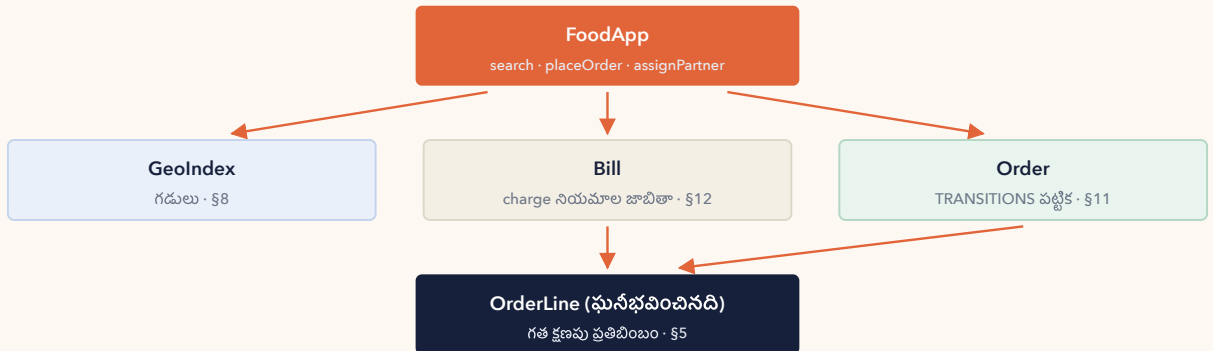
ఇది ఒక సాంకేతిక ఎంపిక కాదు. ఇది ఒక **పన్ను మరియు లెక్కల నిర్ణయం**, మరియు దాన్ని finance/legal team నిర్ణయిస్తుంది. మీ పని ఆ క్రమాన్ని **కనిపించేలా, మార్చగలిగేలా** ఉంచడం – ఒక పెద్ద సూత్రం మధ్యలో దాచడం కాదు.

Interview lo ఈ ఒక్క వాక్యం చెప్పండి: *"The order of the charge rules is itself a business decision, so it's a list, not a formula."* – ఇది మీరు **product గురించి** ఆలోచిస్తున్నారని చూపిస్తుంది.

13. మొత్తం code ఒకే చోట • నడిపి చూద్దాం

నిర్మాణం • మూడు **SYSTEMS**, ఒక **APP**

FoodApp ఈ మూడింటినీ కలుపుతుంది – కానీ ఒక్కొక్కటి విడిగా పనిచేస్తుంది



```

'use strict';
const KM_PER_DEG_LAT = 111.0;
const kmPerDegLon = (lat) => 111.320 * Math.cos(lat * Math.PI / 180);
const distanceKm = (a, b) => Math.hypot((a.lon-b.lon)*kmPerDegLon(a.lat),
                                         (a.lat-b.lat)*KM_PER_DEG_LAT);
const rs = (n) => (n < 0 ? '-₹' : '₹') + Math.abs(n).toFixed(2)
                                     .replace(/\.00$/, '');

// ----- 1 · గత సంఘటన: snapshot -----
class MenuItem { constructor(n, p) { this.name = n; this.price = p; } }

class OrderLine {
  constructor(item, qty) {
    Object.assign(this, { itemId: item.name, name: item.name,
                        price: item.price, qty });
    Object.freeze(this);
  }
  get amount() { return this.price * this.qty; }
}

// ----- 2 · బిల్లు: నియమాల జాబితా -----
class PackagingFee {
  constructor(perItem = 10) { this.perItem = perItem; }
  apply(ctx) {
    return { label: 'Packaging', amount: ctx.lines.length * this.perItem };
  }
}

class DistanceFee {
  constructor(base = 20, perKm = 8) { Object.assign(this, { base, perKm }); }
  apply(ctx) {
    return { label: `Delivery (${ctx.distanceKm.toFixed(1)} km)`,
            amount: this.base + Math.ceil(ctx.distanceKm) * this.perKm };
  }
}

class SurgeFee {
  constructor(active, amount = 25, why = 'rain') {
    Object.assign(this, { active, amount, why });
  }
  apply() {
    return this.active ? { label: `Surge (${this.why})`, amount: this.amount }
                        : null;
  }
}

class Gst {
  constructor(pct = 5) { this.pct = pct; }
  apply(ctx) {
    return { label: `GST ${this.pct}%`,
            amount: +(ctx.runningTotal * this.pct / 100).toFixed(2) };
  }
}

class FlatCoupon {
  constructor(code, off, minOrder = 0) {
    Object.assign(this, { code, off, minOrder });
  }
  apply(ctx) {
    if (ctx.itemTotal < this.minOrder) return null;
    return { label: `Coupon ${this.code}`, amount: -this.off };
  }
}

```

```

}

class Bill {
  constructor(lines, distanceKm, charges) {
    this.lines = lines;
    this.itemTotal = lines.reduce((s, l) => s + l.amount, 0);
    this.rows = [];
    let runningTotal = this.itemTotal;
    for (const c of charges) {
      const row = c.apply({ lines, distanceKm,
                           itemTotal: this.itemTotal, runningTotal });
      if (!row) continue;
      this.rows.push(row);
      runningTotal += row.amount;
    }
    this.total = Math.round(runningTotal * 100) / 100;
    Object.freeze(this);
  }
}

// ----- 3 · స్థితి యంత్రం -----
const TRANSITIONS = {
  PLACED: ['ACCEPTED', 'REJECTED', 'CANCELLED'],
  ACCEPTED: ['PREPARING', 'CANCELLED'],
  PREPARING: ['READY', 'CANCELLED'],
  READY: ['PICKED_UP'],
  PICKED_UP: ['DELIVERED'],
  DELIVERED: [], CANCELLED: [], REJECTED: [],
};

class Order {
  #state = 'PLACED';
  #history = [];
  constructor(id, restaurantName, bill, customer) {
    Object.assign(this, { id, restaurantName, bill, customer });
  }
  get state() { return this.#state; }
  get history() { return this.#history.map(h => h.to); }
  moveTo(next) {
    if (!TRANSITIONS[this.#state].includes(next))
      return { ok: false, reason: `${this.#state} → ${next} అనుమతి లేదు` };
    this.#history.push({ from: this.#state, to: next, at: Date.now() });
    this.#state = next;
    return { ok: true, state: next };
  }
  refundDue() {
    if (this.#state === 'REJECTED') return this.bill.total;
    if (this.#state !== 'CANCELLED') return 0;
    return this.#history.some(h => h.to === 'PICKED_UP')
      ? this.bill.total / 2 : this.bill.total;
  }
}

// ----- 4 · ప్రాదేశిక index -----
class GeoIndex {
  constructor(cellKm = 2, refLat = 17.385) {
    this.cellKm = cellKm;
    this.lonScale = kmPerDegLon(refLat);
    this.cells = new Map();
  }
}

```

```

    }
    #key(lat, lon) {
        return `${Math.floor(lat * KM_PER_DEG_LAT / this.cellKm)}:` +
            `${Math.floor(lon * this.lonScale / this.cellKm)}`;
    }
    add(p) {
        const k = this.#key(p.lat, p.lon);
        if (!this.cells.has(k)) this.cells.set(k, []);
        this.cells.get(k).push(p);
    }
    nearby(point, radiusKm) {
        const span = Math.ceil(radiusKm / this.cellKm);
        const r0 = Math.floor(point.lat * KM_PER_DEG_LAT / this.cellKm);
        const c0 = Math.floor(point.lon * this.lonScale / this.cellKm);
        const out = [];
        for (let dr = -span; dr <= span; dr++)
            for (let dc = -span; dc <= span; dc++)
                for (const p of this.cells.get(`${r0+dr}:${c0+dc}`) ?? [])
                    if (distanceKm(point, p) <= radiusKm) out.push(p);
        return out;
    }
}

class Restaurant {
    constructor(name, lat, lon) {
        Object.assign(this, { name, lat, lon, menu: new Map() });
    }
    add(item) { this.menu.set(item.name, item); return this; }
    item(n) { return this.menu.get(n); }
}

class FoodApp {
    #seq = 0;
    constructor({ charges = [] } = {}) {
        this.index = new GeoIndex(2);
        this.partners = [];
        this.charges = charges;
    }
    addRestaurant(r) { this.index.add(r); return r; }
    addPartner(p) { this.partners.push({ ...p, free: true }); }

    search(point, radiusKm = 3) {
        return this.index.nearby(point, radiusKm)
            .map(r => ({ r, km: distanceKm(point, r) }))
            .sort((a, b) => a.km - b.km);
    }

    placeOrder(customer, restaurant, wanted) { // wanted = [[name, qty], ...]
        const lines = wanted.map(([n, q]) => new OrderLine(restaurant.item(n), q));
        const bill = new Bill(lines, distanceKm(customer, restaurant), this.charges);
        return new Order(`ORD-${++this.#seq}`, restaurant.name, bill, customer);
    }

    assignPartner(restaurant) {
        const free = this.partners.filter(p => p.free);
        if (!free.length) return null;
        const best = free.reduce((a, b) =>
            distanceKm(restaurant, a) <= distanceKm(restaurant, b) ? a : b);
        best.free = false;
        return { name: best.name, km: +distanceKm(restaurant, best).toFixed(1) };
    }
}

```

```
}  
}
```

నడిపి చూద్దాం:

```
const app = new FoodApp({ charges: [  
  new PackagingFee(10),  
  new DistanceFee(20, 8),  
  new SurgeFee(true),  
  new Gst(5),  
  new FlatCoupon('WELCOME50', 50, 250),  
]});
```

```
--- Ravi కి 3 km లోపల ---  
Dosa Corner      0.20 km  
Biryani House    0.45 km
```

```
--- బిల్లు ---  
Chicken Biryani x1      ₹250  
Raita x1                ₹40  
-----  
Item total              ₹290  
Packaging               ₹20  
Delivery (0.4 km)      ₹28  
Surge (rain)           ₹25  
GST 5%                  ₹18.15  
Coupon WELCOME50       -₹50  
-----  
TOTAL                   ₹331.15
```

```
--- ధర మారినా పాత order మారదు ---  
కొత్త menu ధర : ₹300  
order మొత్తం : ₹331.15 ← మారలేదు
```

```
--- ప్రయాణం ---  
partner      : { name: 'Kiran', km: 0.1 }  
ఇప్పటి స్థితి  : PICKED_UP  
Ravi cancel  : { ok: false, reason: 'PICKED_UP → CANCELLED అనుమతి లేదు' }  
చివరికి      : DELIVERED • refund ₹0  
ప్రయాణం     : ACCEPTED → PREPARING → READY → PICKED_UP → DELIVERED
```

ఈ output ని పంక్తి పంక్తిగా చదువుదాం

Dosa Corner 0.20 km – Pizza Point జాబితాలో లేదు, ఎందుకంటే అది 12 km దూరంలో ఉంది. మరియు ఆ గడి అసలు తెరవనే లేదు.

బిల్లు ఆరు వరుసలు – ప్రతి వరుసకీ ఒక పేరు. **Delivery (0.4 km)** label lo దూరం ఉంది; **Surge (rain)** lo కారణం ఉంది. User "ఎందుకు ₹331?" అని అడిగితే జవాబు **బిల్లులోనే** ఉంది.

order మొత్తం : ₹331.15 ← మారలేదు – menu ధర ₹300 అయినా order ₹331.15 గానే ఉంది. §4 lo ఇదే విరిగింది.

PICKED_UP → CANCELLED అనుమతి లేదు – తిరస్కరణ ఒక వస్తువు, మరియు అది **కారణం** చెప్తోంది. §10
lo ఇక్కడ ₹200 నష్టపోయాం; ఇప్పుడు **₹0**.

ACCEPTED → PREPARING → READY → PICKED_UP → DELIVERED – పూర్తి ప్రయాణం నిల్వ ఉంది.
Support team "ఈ order ఎక్కడ ఆగింది?" అని అడిగితే ఇదే జవాబు.

దశల నుంచి ఇక్కడికి – ఏమి చేరింది

ఎక్కడ నుంచి	ఏమి చేరింది	ఎందుకు
§3	MenuItem, Order	మౌలిక అస్థిపంజరం
§4 (ఎరుపు)	OrderLine – ఘనీభవించిన ప్రతిబింబం	ధర మారితే పాత order మారింది
§6	distanceKm + అక్షాంశ సవరణ	తూర్పు-పడమర దూరాలు తప్పు
§7 (ఎరుపు)	GeoIndex – గడులు	2,000 searches కి 8.5 సెకన్లు
§8	స్థిర refLat	గడి వంగిపోయి ఫలితాలు తప్పు
§9 (ఎరుపు)	TRANSITIONS పట్టిక	27 అసాధ్యమైన స్థితులు
§11	#history	refund నియమానికి గతం కావాలి
§12	charges జాబితా	బిల్లు విభజించి చూపించాలి

Part 6 – Interview Io

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

45 నిమిషాల TIME BUDGET

మూడు systems ఉన్నాయి – అన్నిటినీ సమానంగా చెయ్యకండి

6 నిమి

Clarify – ధర మారితే? ఎన్ని restaurants? cancel ఎప్పుడు?

9 నిమి – Order

snapshot vs reference - ₹250 ఉదాహరణ చెప్పండి

12 నిమి – GeoIndex

గడులు · ఇక్కడే ఎక్కువ మంది తేలిపోతారు

10 నిమి – State

TRANSITIONS పట్టిక · refund

8 నిమి

Bill నియమాలు · scale · tracking

ఏమి తప్పక చెప్పాలి

1. **Order ఒక గత సంఘటన** (§5) – MenuItem కి reference కాదు, snapshot. ₹250 → ₹300 ఉదాహరణ చెప్పండి, అది 20 సెకన్లు మరియు అది గుర్తుండిపోతుంది.
2. **స్థానం ఒక index కావాలి** (§8) – గడులు లేదా geohash. మరియు గడి ≈ search వ్యాసార్థం.
3. **ఒకే state + transition పట్టిక** (§11) – booleans కాదు. "27 అసాధ్యమైన స్థితులు" అనే లెక్క చెప్పండి.
4. **Charges ఒక జాబితా** (§12) – మరియు క్రమం ఒక business నిర్ణయం.
5. **Refund కి #history కావాలి** (§11) – ప్రస్తుత స్థితి మాత్రమే సరిపోదు.

ఏమి వదిలేయాలి

- **Partner assignment ని లోతుగా వెళ్ళొద్దు** – "nearest free partner, and the real version is a matching problem" అని చెప్పి ముందుకి వెళ్ళండి (§16).
- **Ratings, reviews, recommendations** – ఇవి వేరే problem. అడిగితేనే.
- **Payment gateway** – "assume a payment service" అని చెప్పండి.
- **Bill io అన్ని charge classes రాయొద్దు** – రెండు రాసి, మిగతావి ఒక వాక్యంలో.

15. నీటితో చెప్పాల్సిన English script

"Before I draw anything – three questions. If a restaurant raises a price, what should a customer's old order show? How many restaurants are we indexing, and over what area? And how late can a customer cancel?

I ask the first one because it decides the core model. The natural thing is for an Order to hold references to MenuItems. That's wrong, and it's wrong in a way that costs money. If the restaurant changes the price from two-fifty to three hundred, the customer's past order silently becomes three hundred, and if they ask for a refund you refund fifty rupees too much. Same for the item name.

An order isn't a view of what's on the menu – it's a record of a past event. So every order line is a frozen snapshot: item id, the name at that moment, the price at that moment, quantity. I'd keep the item id separately from the display name, because reporting needs the id and the receipt needs the name as it was. This is how invoices and bank statements work everywhere.

For nearby search, the obvious version computes distance to every restaurant and filters. I measured it – two thousand searches over two hundred thousand restaurants took about eight and a half seconds, and it grows linearly, so a restaurant opening in Delhi slows down every search in Hyderabad. That's the tell that location is only an attribute, not an index.

So I'd bucket the map into cells, roughly two kilometres square, keyed by row and column. A two-kilometre search only opens the nine cells around you. Same measurement dropped from nine hundred milliseconds to twenty-two at twenty thousand restaurants – and I checked that both versions return identical results before I trusted the speedup. The cell size should be about the search radius; doubling the cell makes each one four times bigger, and I measured that as three to four times slower.

One trap I hit: kilometres per degree of longitude depends on latitude, so if you compute the column using each point's own latitude the grid skews and you lose restaurants. Use one reference latitude for the whole city.

For order state, the tempting model is a boolean per stage – accepted, picked up, delivered, cancelled. That lets an order be cancelled and delivered at the same time, which means you refund half the money for food that was eaten. Five booleans give thirty-two combinations and only about five are real, so twenty-seven of them exist purely as places for bugs to live. I'd use one state field plus a table of allowed transitions. Picked-up to cancelled simply isn't in the table, so that's not a rule in an if-statement – it's a fact about the system. And I'd keep the transition history, because the refund rule needs to know whether the order was ever picked up, not just where it is now.

Last piece: the bill. I wouldn't write one formula. Each charge – packaging, distance, surge, GST, coupon – is a small object returning a labelled row, and the bill is the list of rows. That's what the app actually shows the user. And the order of that list matters: applying GST before the coupon

versus after changes the total by about two and a half rupees, which at a hundred thousand orders a day is two and a half lakh. That's a finance decision, so it should be visible and configurable rather than buried."

16. Follow-ups – tracking, scale, ratings

Follow-up	జవాబు	మార్గ classes
"కొత్త charge – late night fee"	కొత్త charge class, జాబితాలో చేర్చడం	+1 కొత్తది, 0 edits
"Coupon 10% off, గరిష్ఠం ₹100"	కొత్త PercentCoupon	+1 కొత్తది
"Restaurant order ని reject చేస్తే?"	REJECTED ఇప్పటికే పట్టికలో ఉంది	0
"ఒక partner కి రెండు orders (batching)"	assignPartner → capacity + route	FoodApp
"Search lo veg/rating/cuisine filters"	nearby ఫలితం మీద filter	0 concepts
"Notifications – 'your order is ready'"	Deep Dive 07 event bus, state మారినప్పుడు publish	0 concepts
"Live tracking"	కింద చూడండి	కొత్త system
"10 కోట్ల orders"	కింద చూడండి	Storage layer

Live tracking – ఇది వేరే system

"ఇది order system కాదు. Partner phone ప్రతి కొన్ని సెకన్లకే స్థానం పంపుతుంది – అంటే ఇది అధిక-వేగపు రాత, తక్కువ విలువ కలిగిన డేటా.

దాన్ని orders database lo రాయకూడదు. ఒక in-memory store (Redis) lo partnerId → {lat, lon, at} ఉంచి, customer app కి WebSocket ద్వారా పంపడం. ఆ డేటా delivery అయ్యాక అవసరం లేదు.

కీలక వాక్యం: order డేటా మరియు location డేటా వేర్వేరు జీవితకాలాలు, వేర్వేరు స్థిరత్వ అవసరాలు కలిగినవి – కాబట్టి వేర్వేరు నిల్వలు."

Scale

"ఇప్పుడు అంతా memory lo ఉంది. నిజమైన system lo:

- **Orders** – ఒక సాధారణ database, కానీ **OrderLine** lo ధరలు నిల్వ చేసి ఉండటం వల్ల **joins** అవసరం లేదు receipt చూపించడానికీ అది ఒక పెద్ద లాభం.
- **GeoIndex** – ఇది memory lo ఉండాలి, మరియు **నగరం వారీగా విభజించాలి**. Hyderabad server Hyderabad గడులని మాత్రమే పట్టుకుంటుంది. నిజ ప్రపంచంలో దీనికి **geohash** లేదా **S2 cells** వాడతారు – నా గడుల ఆలోచనే, కానీ ప్రమాణీకరించినది.
- **Restaurant menu** – ఇది ఎక్కువగా చదివేది, తక్కువగా మారేది → cache. Deep Dive 02 lo చూసిన LRU ఇక్కడ సరిగ్గా సరిపోతుంది.

మరియు గమనించండి – **snapshot నిర్ణయం (§5) scale ని కూడా సులభం చేసింది**. Order ఏ live డేటా మీదా ఆధారపడదు, కాబట్టి దాన్ని ఎక్కడైనా, ఎప్పుడైనా చదవొచ్చు."

17. ఏమి నేర్చుకున్నాం

ఆలోచన	ఇక్కడ ఎలా కనిపించింది	ఇంకెక్కడ వస్తుంది
గత సంఘటన = snapshot	OrderLine ఘనీభవించినది (§5)	Invoices, payslips, audit logs, event sourcing
వెతకొద్దు, ఎక్కడో లెక్కించు	GeoIndex గడులు (§8)	Deep Dive 08 (HashMap), Deep Dive 10 §8
Index కీ స్థిరంగా ఉండాలి	refLat ఉచ్చు (§8)	Deep Dive 08 – mutable keys
అసాధ్యమైన స్థితిని వ్యక్తపరచలేని design	32 → 5 (§10)	ప్రతి state machine, prop types, DB constraints
నియమాలు ఒక జాబితా, ఒక సూత్రం కాదు	charges (§12)	Pricing, permissions, validation pipelines
క్రమం ఒక business నిర్ణయం	GST vs coupon (§12)	Middleware, filters, discount stacking
తిరస్కరణ ఒక API	{ ok, reason } (§11)	Deep Dive 03 §12, Deep Dive 10 §13
వేగం కొలిచే ముందు సరైనత పోల్చు	§8 lo ఆ bug దీరికింది	ప్రతి optimisation

ఒక చివరి ఆలోచన - ఈ DOC LO అత్యంత విలువైనది

§8 lo నేను గడుల version రాసి, వేగం కొలిచాను - **12x వేగం!** అని రాయబోయాను.

కానీ ముందు ఒక చిన్న తనిఖీ పెట్టాను: "రెండు పద్ధతులూ ఒకే ఫలితాలు ఇస్తున్నాయా?" - జవాబు **లేదు**. ఆ 12x వేగం, **తప్పు జవాబులతో** వచ్చినది.

వేగవంతమైన తప్పు జవాబు, నెమ్మదైన సరైన జవాబు కంటే హీనం. ఏదైనా optimise చేసినప్పుడు - ముందు పాత పద్ధతితో పోల్చి, ఆపై గడియారం చూడండి. Interview lo ఈ అలవాటుని చెప్పడం కూడా విలువైనది.

ఇక్కడి నుంచి ఎక్కడికి

ఈ series lo ఇప్పటివరకు: **01 Parking Lot · 02 Cache · 03 Rate Limiter · 04 BookMyShow · 05 Splitwise · 06 Elevator · 07 Pub-Sub · 08 HashMap · 09 Chess · 10 Meeting Scheduler · 11 Food Delivery.**

వేగవంతమైన revision కోసం - [LLD_Design_Problems_Telugu.pdf](#) .

Food Delivery - అడుగు అడుగునా · ఈ doc lo ఉన్న ప్రతి output, ప్రతి సమయం నిజంగా `node` lo run చేసి తీసినదే 